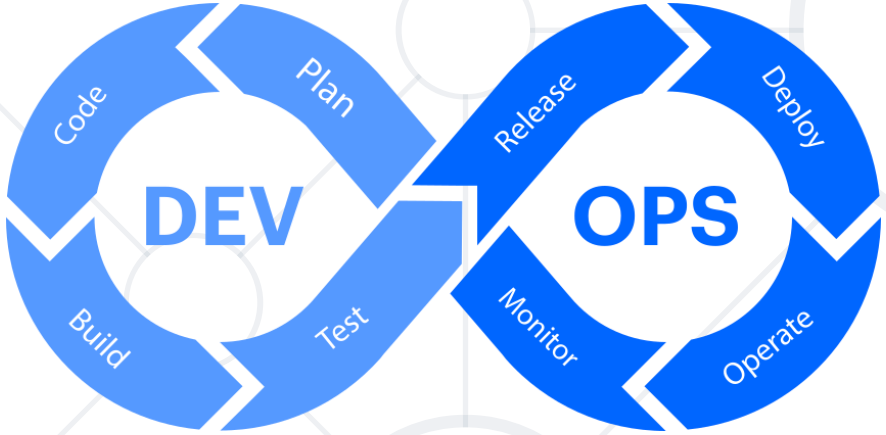
# Transition to GitOps

## Automating Infrastructure Reconciliation




**SoftUni Team**

**Technical Trainers**

Software University

SoftUni

Software University

https://softuni.bg

# Have a Question?

Software University

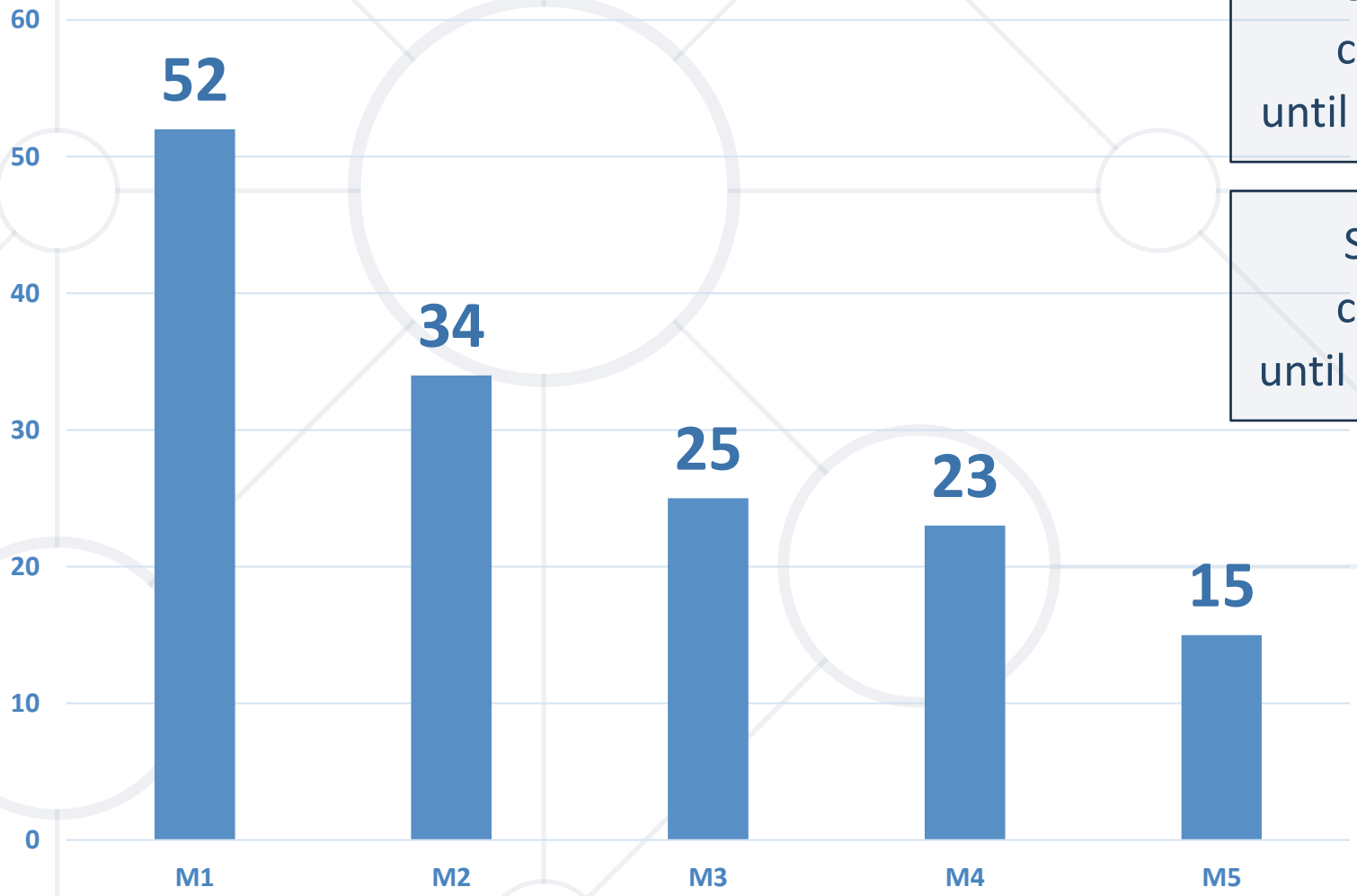
sli.do

#DevOps-CI-CD

facebook.com/groups/

cicdmonitoringjanuary2026

# Homework Progress




Solutions for M5
can be submitted
until 23:59 on 25.02.2026

Solutions for M6
can be submitted
until 23:59 on 04.03.2026

# Previous Module (M5)

## Quick Overview

# What We Covered

1. From DevOps to DevSecOps
2. Key Principles and Practices
3. Common Tools
4. Secure CI/CD Pipeline

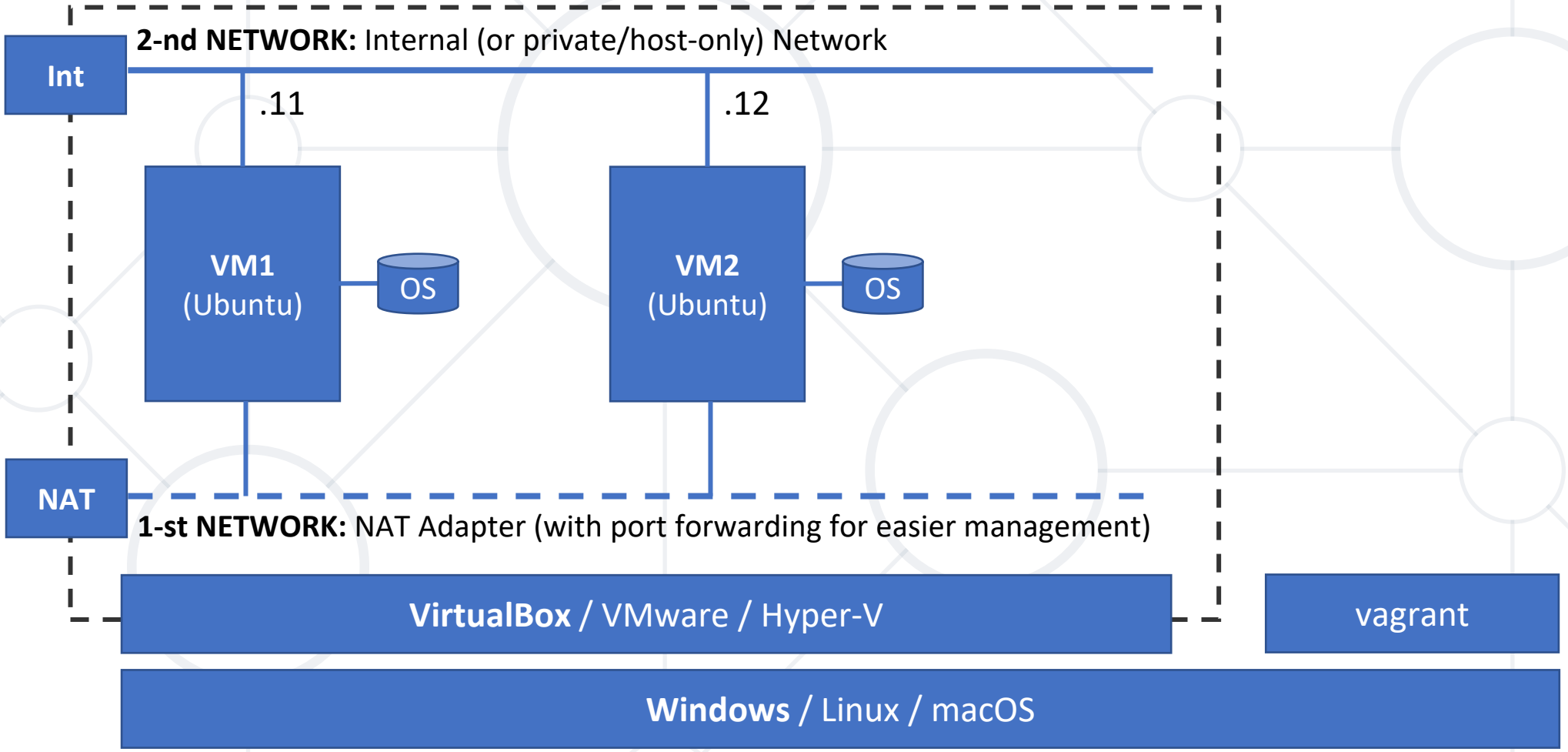

# This Module (M6)

## Topics and Infrastructure

# Table of Contents



# Lab Infrastructure




# The GitOps Philosophy

## Paradigm Shift

# The Road to GitOps: DevOps

Software University

DevOps is a **set of practices** that combines **software development** and **IT operations** to deliver software solutions more **quickly**, **reliably**, and **stably**

**Core Principles**

- Culture
- Automation
- Continuous improvement
- Platform design

**Key Practices**

- Continuous Integration / Continuous Delivery
- Infrastructure as Code
- Microservices
- Platform engineering

# What is GitOps

Software University

**GitOps = Git + Operations**

GitOps is a set of (DevOps) practices

for managing **infrastructure and applications**

using **Git** as the **single source of truth**

# GitOps Principles

- **Declarative**

  A system managed by GitOps must have its **desired state expressed declaratively**

- **Versioned and Immutable**

  Desired state is stored in a way that enforces **immutability**, **versioning** and retains a **complete version history**

- **Pulled Automatically**

  Software agents automatically pull the desired state declarations from the source

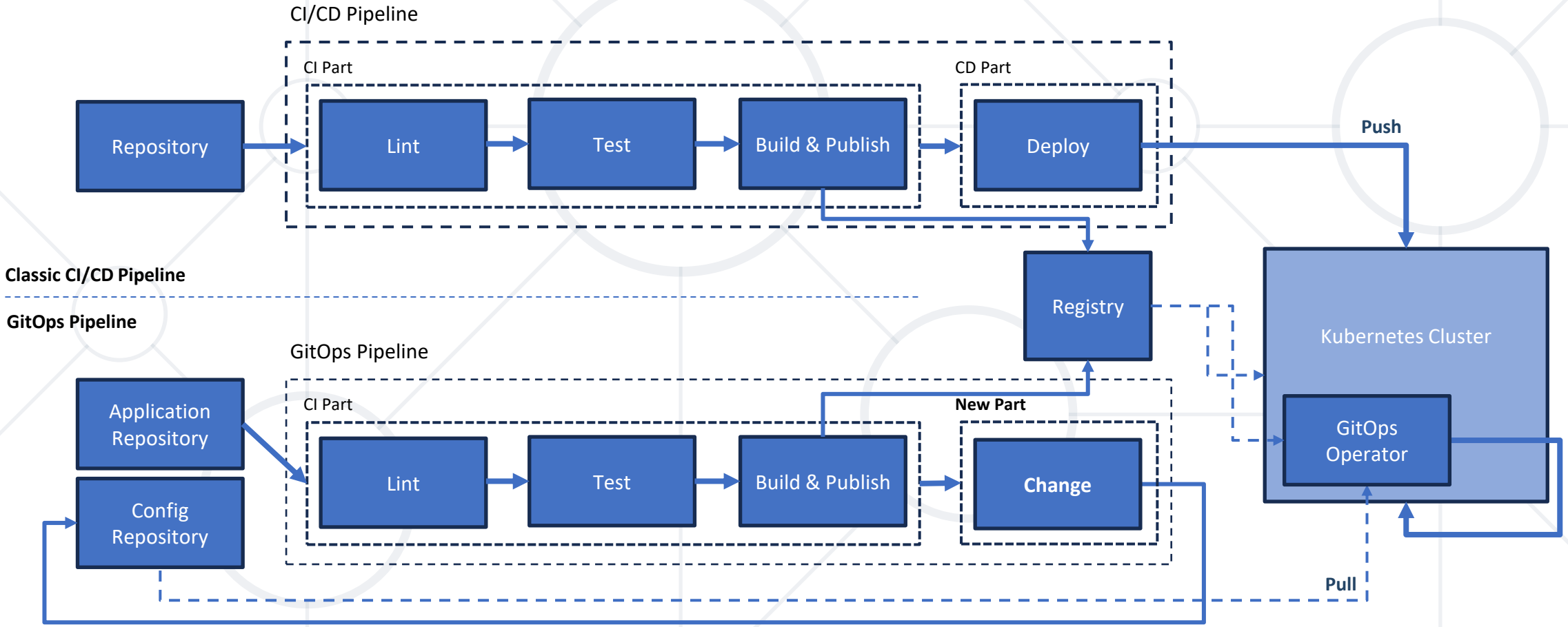
- **Continuously Reconciled**

  Software agents continuously observe actual system state and attempt to apply the desired state

# Push vs Pull

- **The Push Model**
  - Jenkins/Gitea runs a script, connects to the cluster, and executes a command
  - If the cluster changes manually later, the pipeline doesn't know
- **The Pull Model**
  - An agent inside the cluster monitors a Git repo
  - If Git repo changes, the cluster updates
  - If the cluster is tampered with, an agent heals it back to the Git state

# Pipeline Evolution



# Key Benefits

- Faster releases through automation
- Safer operations thanks to versioned audits
- Easy rollbacks by reverting Git commits
- Consistent environments across dev, test, and production
- Improved security by minimizing direct access to systems

# Typical Challenges

- Steep learning curve
- Tooling fragmentation
- Security concerns
- Cultural shift

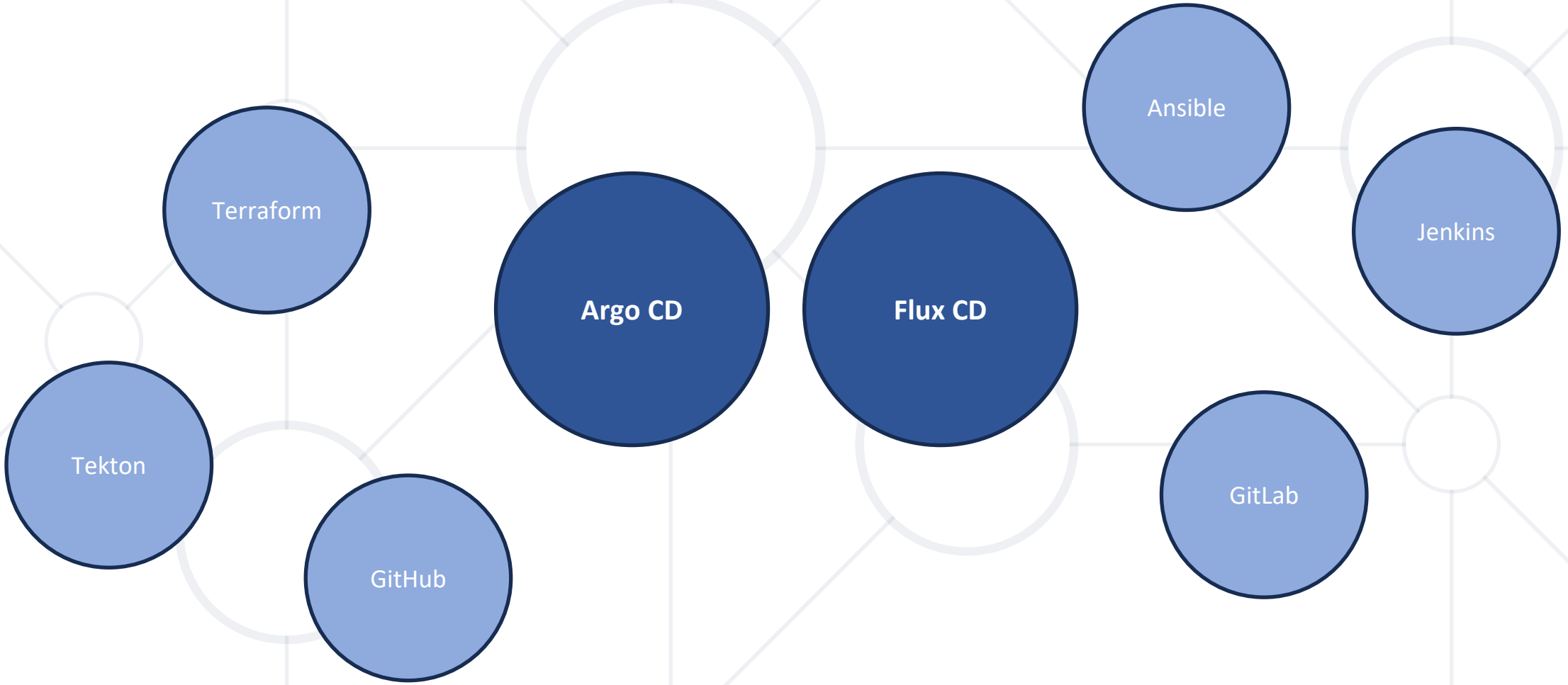

# Practice: Exploration and Preparation

## Explore and Extend the Playground


# Tooling and Installation

## Explore the GitOps Tolling Landscape

# The GitOps Universe *



# Argo CD

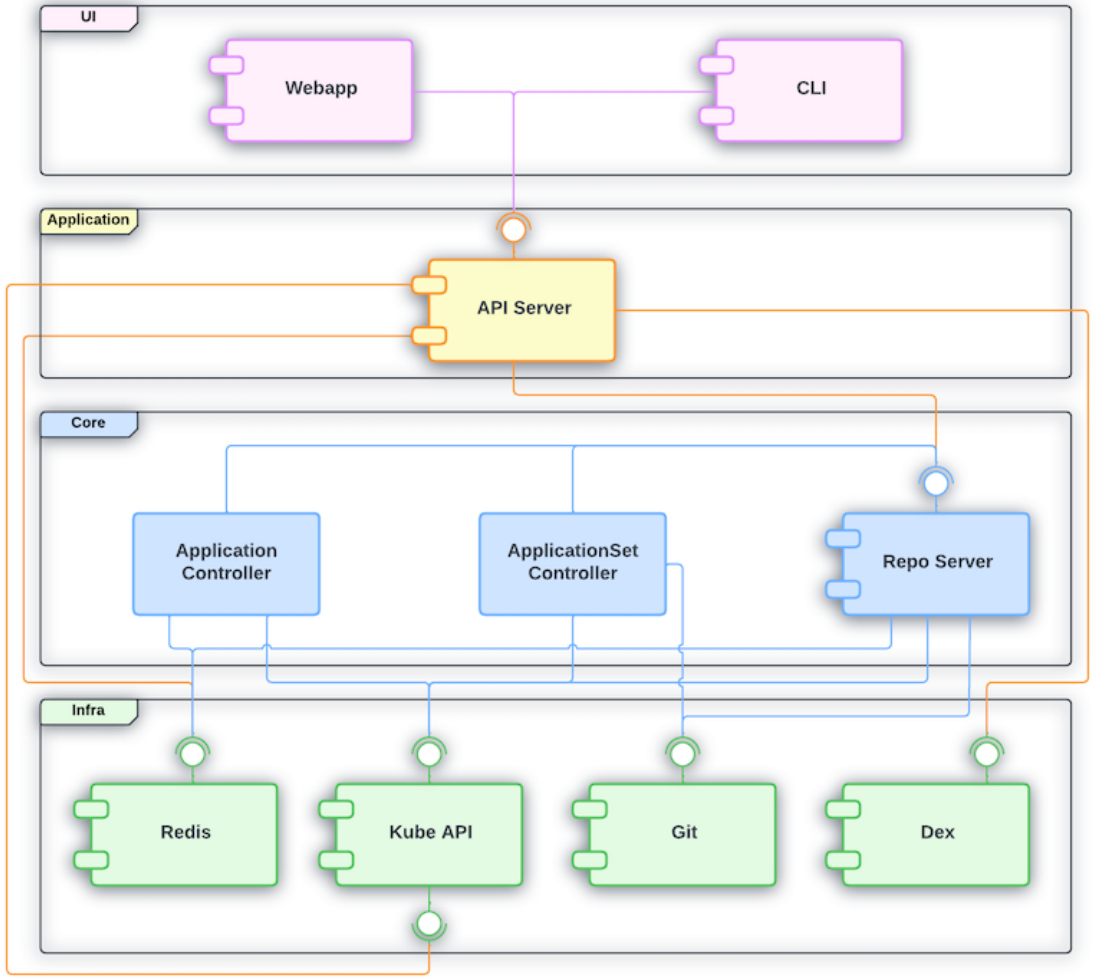
- Declarative application definitions
- Automated synchronization
- Real-time application status monitoring
- Role-based access control (RBAC)
- Multi-cluster support
- Web UI and CLI
- Created by Intuit, now part of CNCF

# Argo CD Architecture

- External components
  - Redis
  - Kubernetes API
  - Git
  - Dex
- Internal components
  - Application Controller
  - ApplicationSet Controller
  - Repo Server
  - API Server
  - Webapp and CLI

ArgoCD relies on those components

Based on the installation profile, some or all of those are being installed



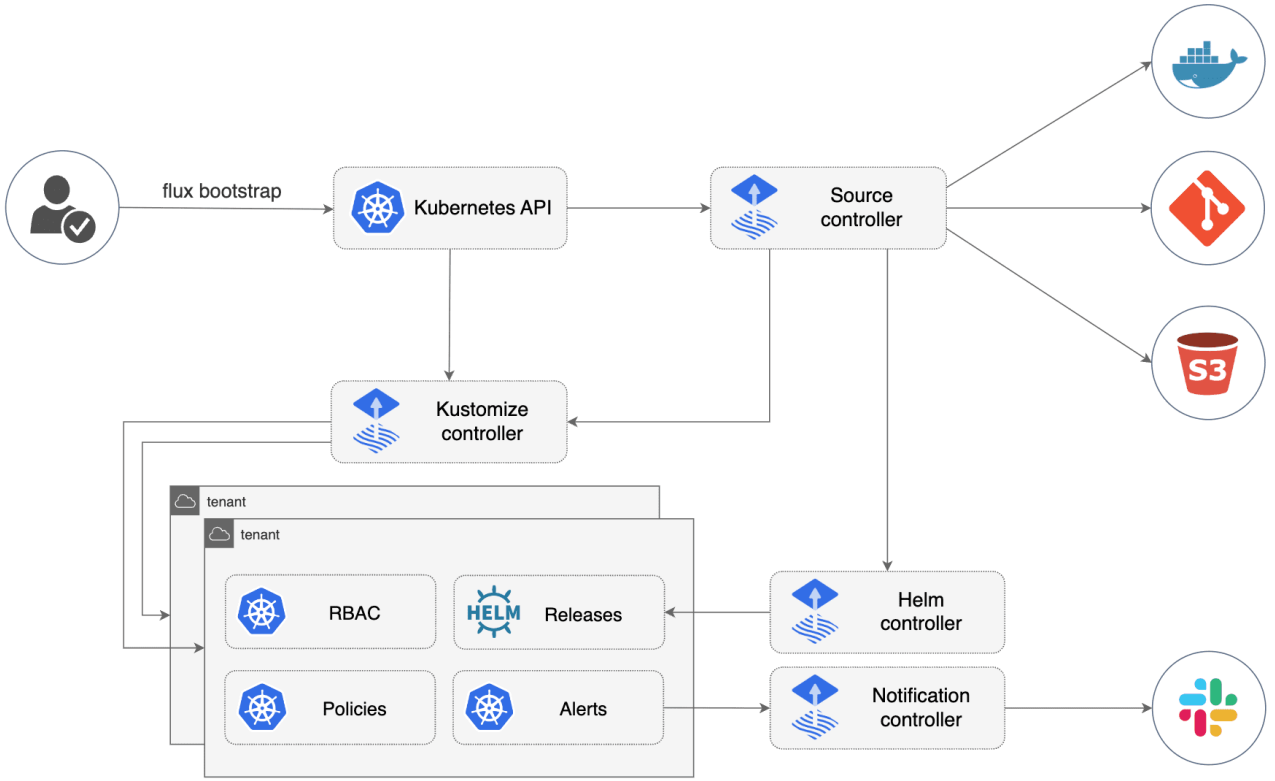
# Flux CD

Software University

- Kubernetes RBAC support
- Supports image patches and updates automation
- A multi-tenant enabled architecture
- Compatible with Kubernetes Cluster API
- A rich ecosystem of extensions, interfaces, and platforms
- Created by Weaveworks, now part of CNCF

# Flux CD Architecture

- Source Controller
- Kustomize Controller
- Helm Controller
- Notification Controller
- Image Automation Controller



# Argo CD vs Flux CD

Software University

| Feature | Argo CD | Flux CD |
|---|---|---|
| CNCF Status | Graduated | Graduated |
| Design Architecture | Complete application | Modular and extensible toolkit |
| User Interaction | Built-in web UI, CLI available | CLI-first, optional lightweight web UI |
| Configuration Sources | Git, Kustomize, Helm | Git, Kustomize, Helm, OCI, S3 |
| Reconciliation Process | Manual and automatic syncs | Manual and automatic syncs |
| Progressive Delivery | Argo Rollouts | Flagger |
| Access Management | Local users, SSO, built-in RBAC | Kubernetes identity and RBAC |
| Extensibility | Limited customization options | Good, third-party integrations |
| Ease of Use | Easy | Moderate |


# Practice: Argo CD on Stage

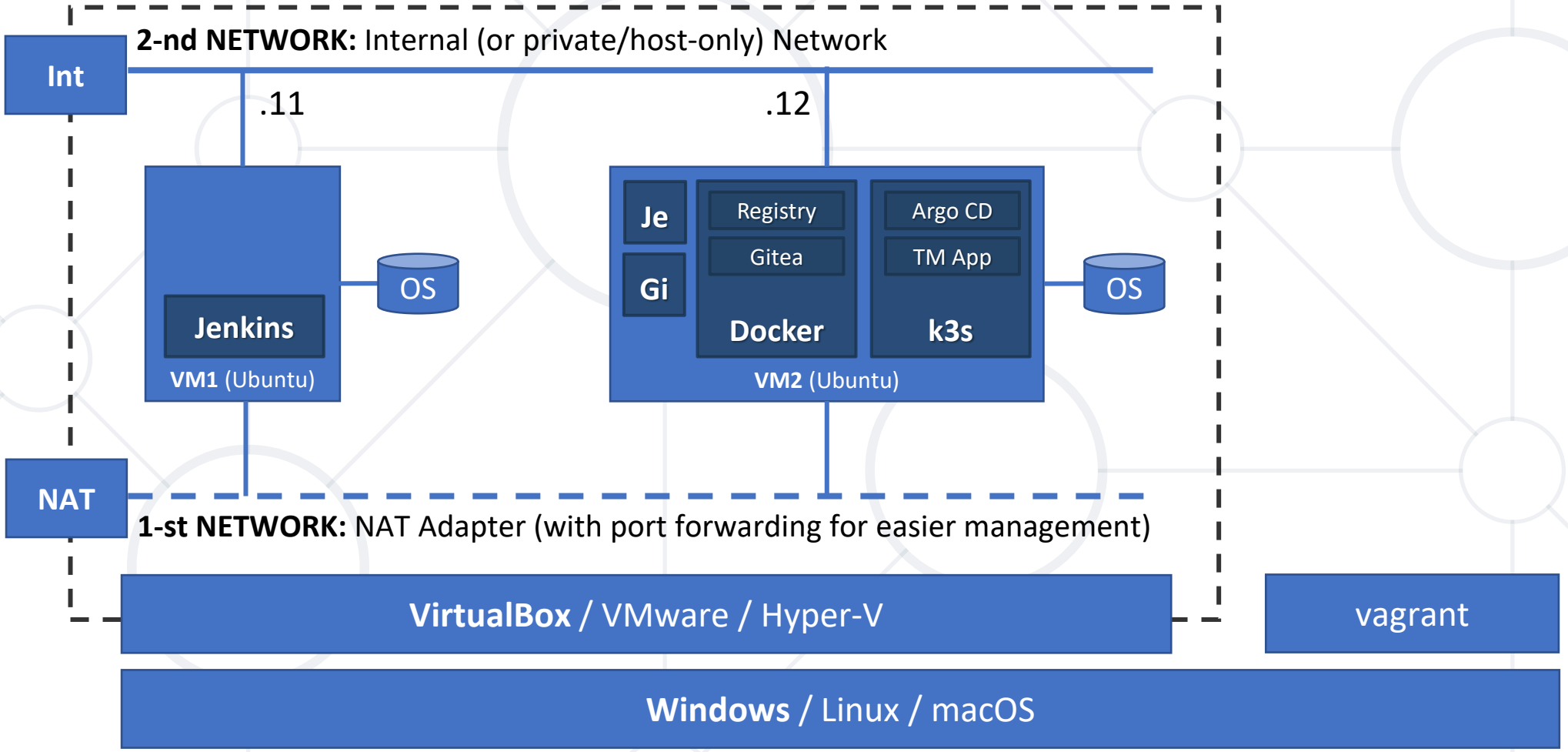
## Setup and First Steps


# Operationalizing GitOps

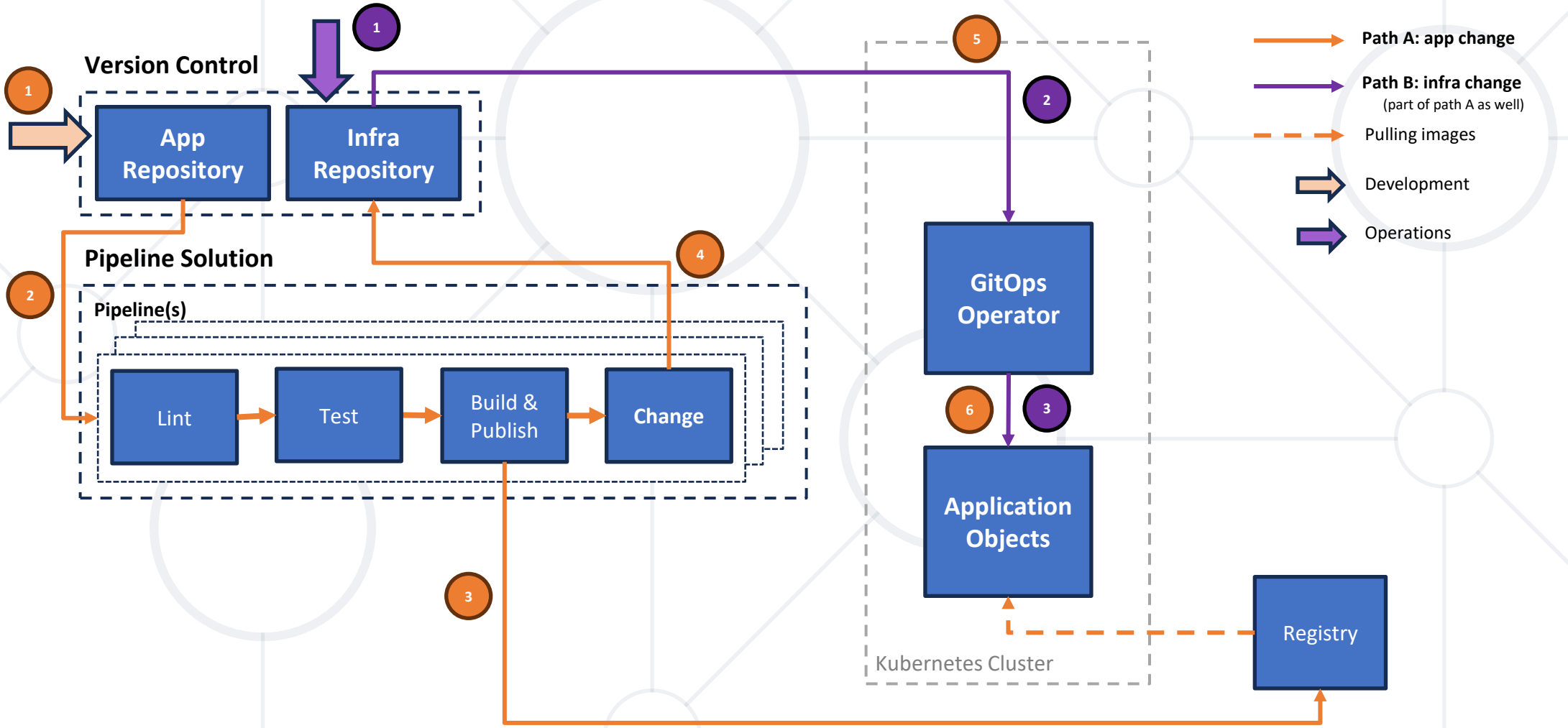
# Operational Features *

- **Auto-Sync**
  - Push to Git, cluster updates instantly
- **Pruning**
  - Automatically deleting resources that were removed from Git
- **Self-Healing**
  - Automatically overwriting manual changes
- **Sync Phases (and Hooks)**
  - Define when resources are applied such as before or after the main sync operation
- **Sync Waves**
  - An alternative method of changing the sync order of resources

# Our Environment: Infrastructural View



# Our Environment: Logical View




# Practice: Argo CD in Action

## Let’s Move Even Further

# Questions?

SoftUni

Software University | SoftUni AI | SoftUni Digital | SoftUni Creative | SoftUni Foundation | СофтУни БУДИТЕЛ | Finance Academy

# Trainings @ Software University (SoftUni)

- Software University – High-Quality Education, Profession and Job for Software Developers
  - softuni.bg, about.softuni.bg
- Software University Foundation
  - softuni.foundation
- Software University @ Facebook
  - facebook.com/SoftwareUniversity

# License

- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – https://about.softuni.bg
- © Software University – https://softuni.bg